AutoDeploy Hub - Step-by-Step Setup Guide

Complete Guide to Running the Project

This guide provides detailed, step-by-step instructions to get AutoDeploy Hub running on your Windows machine.

Prerequisites Checklist

Before starting, ensure you have:

- [ ] Windows 10/11 (64-bit)
- [ ] Administrator access to your computer
- [ ] Stable internet connection
- [ ] At least 10 GB free disk space

Part 1: Installing Required Software

Step 1: Install Docker Desktop

1.1 Download Docker Desktop

1. Open your web browser
2. Go to: https://www.docker.com/products/docker-desktop
3. Click the **"Download for Windows"** button
4. Wait for the download to complete (~500 MB)

1.2 Install Docker Desktop

1. Locate the downloaded file: `Docker Desktop Installer.exe`
2. **Right-click** on it and select **"Run as administrator"**
3. Click **"Yes"** when prompted by User Account Control
4. In the installer:
    - Check **"Use WSL 2 instead of Hyper-V"** (recommended)
    - Click **"OK"**
5. Wait for installation to complete (5-10 minutes)
6. Click **"Close and restart"** when prompted
7. Your computer will restart

1.3 Start Docker Desktop

1. After restart, Docker Desktop should start automatically
2. If not, search for **"Docker Desktop"** in Windows Start menu
3. Click to open it
4. Wait for Docker to start (you'll see "Docker Desktop is running" in the system tray)
5. Accept the Docker Subscription Service Agreement if prompted

1.4 Verify Docker Installation

1. Open **Command Prompt** or **PowerShell**:
    - Press `Windows + R`
    - Type `cmd` and press Enter
2. Type the following command and press Enter:

```
docker --version
```

3. You should see output like: `Docker version 24.0.x, build xxxxx`
4. Type this command:

```
docker-compose --version
```

5. You should see output like: `Docker Compose version v2.x.x`

If both commands work, Docker is installed correctly! ☑

Step 2: Install Git (Optional but Recommended)

2.1 Download Git

1. Go to: https://git-scm.com/download/win
2. Download will start automatically
3. Wait for download to complete (~50 MB)

2.2 Install Git

1. Run the downloaded installer: `Git-x.xx.x-64-bit.exe`
2. Click **"Next"** through the installation wizard
3. Use default settings (just keep clicking "Next")
4. Click **"Install"**
5. Click **"Finish"** when done

2.3 Verify Git Installation

1. Open a new Command Prompt window
2. Type:

```
git --version
```

3. You should see: `git version 2.x.x`

Git is installed! ☑

Part 2: Running AutoDeploy Hub

Step 3: Navigate to Project Directory

3.1 Open Command Prompt

1. Press `Windows + R`
2. Type `cmd` and press Enter

3.2 Go to Project Folder

1. Type the following command and press Enter:

```
cd c:\Users\Vignesh\Desktop\autodeploy-hub-main
```

2. You should now be in the project directory

3.3 Verify You're in the Right Place

1. Type:

```
dir
```

2. You should see files like:
   - `docker-compose.yml`
   - `package.json`
   - `README.md`
   - `backend` (folder)
   - `src` (folder)

You're in the right directory! ☑

Step 4: Start the Application

4.1 Build and Start Services

1. Make sure Docker Desktop is running (check system tray)
2. In the Command Prompt, type:

```
docker-compose up --build
```

3. Press Enter

4.2 Wait for Build to Complete

You will see a lot of output. This is normal! The process includes:

1. **Downloading base images** (first time only)
   - You'll see: `Pulling backend...`, `Pulling frontend...`
   - This can take 5-10 minutes on first run
2. **Installing dependencies**
   - You'll see: `npm install` running
   - This installs all required packages
3. **Building containers**
   - You'll see: `Building backend...`, `Building frontend...`
   - Docker creates container images
4. **Starting services**
   - You'll see: `Creating autodeploy-backend...`
   - You'll see: `Creating autodeploy-frontend...`

4.3 Know When It's Ready

The application is ready when you see:

```
autodeploy-backend  | ╔═══════════════════════════════════════╗
autodeploy-backend  | ║   AutoDeploy Hub Backend Server       ║
autodeploy-backend  | ║   Status: Running ✓                   ║
autodeploy-backend  | ║   Port: 5000                          ║
autodeploy-backend  | ╚═══════════════════════════════════════╝
```

Application is running! ☑

Step 5: Access the Application

5.1 Open Frontend in Browser

1. Open your web browser (Chrome, Edge, Firefox, etc.)
2. In the address bar, type:

```
http://localhost:3000
```

3. Press Enter
4. You should see the **AutoDeploy Hub** homepage

5.2 Test Backend Connection

1. On the webpage, scroll down to the **"Live Demo"** section
2. Click the **"Check Backend Health"** button
3. Wait 1-2 seconds
4. You should see a green success box with:

```
{
  "status": "ok",
  "message": "Backend is running",
  "time": "2026-01-08T05:37:20.123Z"
}
```

Frontend and Backend are communicating! ☑

5.3 Access Backend Directly (Optional)

1. Open a new browser tab
2. Go to:

```
http://localhost:5000/health
```

3. You should see the same JSON response

Step 6: Verify Everything is Working

6.1 Check Container Status

1. Open a **new** Command Prompt window (keep the first one running)
2. Type:

```
docker ps
```

3. You should see two containers:
   - `autodeploy-backend` - Status: Up (healthy)
   - `autodeploy-frontend` - Status: Up (healthy)

6.2 Check Logs

1. To see backend logs:

```
docker-compose logs backend
```

2. To see frontend logs:

```
docker-compose logs frontend
```

3. To see all logs in real-time:

```
docker-compose logs -f
```

(Press `Ctrl + C` to stop viewing logs)

Everything is working correctly! ☑

Part 3: Managing the Application

Step 7: Stopping the Application

Method 1: Graceful Shutdown

1. Go to the Command Prompt window where `docker-compose up` is running
2. Press `Ctrl + C`
3. Wait for services to stop gracefully
4. You'll see: `Stopping autodeploy-backend...`, `Stopping autodeploy-frontend...`

Method 2: Using Docker Compose Down

1. Open Command Prompt
2. Navigate to project directory:

```
cd c:\Users\Vignesh\Desktop\autodeploy-hub-main
```

3. Type:

```
docker-compose down
```

4. This stops and removes containers

Application stopped! ☑

Step 8: Restarting the Application

8.1 Quick Restart (Without Rebuild)

If you haven't changed any code:

```
cd c:\Users\Vignesh\Desktop\autodeploy-hub-main
docker-compose up -d
```

The `-d` flag runs containers in the background (detached mode).

8.2 Full Restart (With Rebuild)

If you changed code or want to rebuild:

```
cd c:\Users\Vignesh\Desktop\autodeploy-hub-main
docker-compose up --build
```

8.3 Check if Running

```
docker-compose ps
```

You should see both services with "Up" status.

Part 4: GitHub Setup (Optional - For CI/CD)

Step 9: Create GitHub Repository

9.1 Create Repository on GitHub

1. Go to: https://github.com
2. Sign in to your account (or create one)
3. Click the **"+"** icon in top right
4. Select **"New repository"**
5. Enter repository name: `autodeploy-hub`
6. Choose **"Public"** or **"Private"**
7. **Do NOT** check "Initialize with README"
8. Click **"Create repository"**

9.2 Note Your Repository URL

You'll see a URL like:

```
https://github.com/YOUR_USERNAME/autodeploy-hub.git
```

Copy this URL - you'll need it next.

Step 10: Push Code to GitHub

10.1 Initialize Git Repository

1. Open Command Prompt
2. Navigate to project:

```
cd c:\Users\Vignesh\Desktop\autodeploy-hub-main
```

3. Initialize git:

```
git init
```

10.2 Add All Files

```
git add .
```

10.3 Commit Files

```
git commit -m "Initial commit: AutoDeploy Hub - Production Ready"
```

10.4 Add Remote Repository

Replace `YOUR_USERNAME` with your GitHub username:

```
git remote add origin https://github.com/YOUR_USERNAME/autodeploy-hub.git
```

10.5 Push to GitHub

```
git branch -M main
git push -u origin main
```

You may be prompted to sign in to GitHub. Enter your credentials.

Code is now on GitHub! ☑

Step 11: View CI/CD Pipeline

11.1 Access GitHub Actions

1. Go to your repository on GitHub
2. Click the **"Actions"** tab at the top
3. You should see a workflow running: "AutoDeploy Hub CI/CD"

11.2 View Workflow Details

1. Click on the workflow run
2. You'll see all the steps:
    - ☑ Checkout code
    - ☑ Set up Docker Buildx
    - ☑ Build backend Docker image
    - ☑ Build frontend Docker image
    - ☑ Start services with Docker Compose
    - ☑ Wait for services to be healthy
    - ☑ Test backend health endpoint
    - ☑ Test frontend accessibility
    - ☑ Show container logs
    - ☑ Cleanup

11.3 Verify Success

All steps should have green checkmarks ☑

CI/CD is working! ☑

Part 5: Troubleshooting

Common Issues and Solutions

Issue 1: "Docker daemon is not running"

Solution:

1. Open Docker Desktop application
2. Wait for it to start (whale icon in system tray)
3. Try your command again

Issue 2: "Port 3000 is already in use"

Solution:

1. Find what's using the port:

    ```
    netstat -ano | findstr :3000
    ```

2. Note the PID (last column)
3. Kill the process:

    ```
    taskkill /PID <PID> /F
    ```

4. Or change the port in `docker-compose.yml`

Issue 3: "Port 5000 is already in use"

Solution:
Same as above, but use `:5000` instead of `:3000`

Issue 4: "Cannot connect to backend"

Solution:

1. Check if backend is running:

    ```
    docker-compose ps
    ```

2. Check backend logs:

    ```
    docker-compose logs backend
    ```

3. Restart backend:

    ```
    docker-compose restart backend
    ```

Issue 5: "Build failed"

Solution:

1. Stop all containers:

```
docker-compose down -v
```

2. Clear Docker cache:

```
docker system prune -a
```

3. Rebuild:

```
docker-compose up --build
```

Issue 6: "Frontend shows blank page"

Solution:

1. Check browser console (F12)
2. Check frontend logs:

```
docker-compose logs frontend
```

3. Clear browser cache and reload

Part 6: Daily Usage

Starting Your Day

```
# 1. Open Command Prompt
# 2. Navigate to project
cd c:\Users\Vignesh\Desktop\autodeploy-hub-main

# 3. Start application
docker-compose up -d

# 4. Open browser to http://localhost:3000
```

Ending Your Day

```
# 1. Stop application
docker-compose down
```

Making Changes

```
# 1. Stop application
docker-compose down

# 2. Make your code changes

# 3. Rebuild and start
docker-compose up --build
```

Quick Reference Commands

Essential Commands

```
# Start application (background)
docker-compose up -d

# Start application (foreground, see logs)
docker-compose up

# Stop application
docker-compose down

# Rebuild and start
docker-compose up --build

# View logs
docker-compose logs -f

# Check status
docker-compose ps

# Restart a service
docker-compose restart backend
docker-compose restart frontend
```

Docker Commands

```
# List running containers
docker ps

# List all containers
docker ps -a

# View container logs
docker logs autodeploy-backend
docker logs autodeploy-frontend

# Execute command in container
docker exec -it autodeploy-backend sh

# Remove all stopped containers
docker container prune

# Remove all unused images
docker image prune -a
```

Success Checklist

After following this guide, you should have:

- [x] Docker Desktop installed and running
- [x] Git installed (optional)
- [x] Application running on http://localhost:3000
- [x] Backend responding on http://localhost:5000
- [x] Frontend successfully calling backend API
- [x] Both containers showing "healthy" status
- [x] Code pushed to GitHub (optional)
- [x] CI/CD pipeline running on GitHub (optional)

What You've Accomplished

Congratulations! You have successfully:

1. ☑ Installed all required software
2. ☑ Started a multi-container Docker application
3. ☑ Verified frontend-backend communication
4. ☑ Learned Docker Compose commands
5. ☑ Set up CI/CD pipeline (if you did GitHub setup)

You now have a **fully functional, production-ready application** running on your machine!

Next Steps

Learn More

- Explore the code in `backend/server.js`
- Modify the frontend in `src/components/HealthDemo.tsx`
- Add new API endpoints
- Customize the UI

Extend the Project

- Add a database (PostgreSQL, MongoDB)
- Add user authentication
- Create more API endpoints
- Add automated tests

Deploy to Production

- Deploy to AWS, Azure, or Google Cloud
- Set up domain name
- Configure HTTPS
- Add monitoring and logging

Getting Help

If you encounter issues:

1. Check the **Troubleshooting** section above
2. Review the `README.md` file in the project
3. Check Docker Desktop logs
4. Review GitHub Actions logs (if using CI/CD)
5. Ensure all prerequisites are installed correctly

Summary

This guide walked you through:

- Installing Docker Desktop and Git

- Running the AutoDeploy Hub application
- Verifying everything works correctly
- Managing the application (start/stop/restart)
- Setting up GitHub and CI/CD (optional)
- Troubleshooting common issues

Your AutoDeploy Hub is now fully operational! 🚀

Enjoy learning DevOps with this production-ready application!